



WEB班ワークショップ説明資料案3（穴埋め問題で能動学習版）



今日のゴール

穴埋め問題を解きながら、タスク管理アプリを段階的に完成させて、CSS装飾とJavaScript動作の実装力を身につける



穴埋め問題形式について

- ⚡ ただの写経ではありません！
- 🧠 自分で考えて空欄を埋めることで理解を深めます
- 🗨️ ペアで話し合いながら最適解を見つけます
- 💡 なぜその答えなのかを説明できるようになります



ワークショップの流れ

Step 0: 完成形を体験しよう（10分）

まずは今日作るアプリを実際に触ってみましょう！

やること：

1. 完成形のタスク管理アプリを操作
2. どんな要素・機能があるか観察
3. 「これを作るには何が必要？」を考える

Step 1: HTML穴埋めチャレンジ（30分）



問題1-1: 基本構造穴埋め（10分）

以下のHTMLの空欄を埋めてください：

```
<!DOCTYPE html>
<html lang="ja">
<head>
```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>学習タスク管理</title>
<link rel="stylesheet" href="_____.css">
</head>
<body>
  <!-- ヘッダー部分 -->
  <_____ class="_____-header">
    <h1><img alt="book icon" data-bbox="198 183 218 198"/> 今日の学習タスク</h1>
    <p>目標: CSS装飾とJavaScript動きをマスターしよう!</p>
  </_____>
</body>
</html>
```

ペアで考えよう:

1. _____ の1番目に入るファイル名は?
2. _____ の2番目に入るHTMLタグは?
3. _____-header の最初の部分は?

選択肢:

- A) div, main, header
- B) section, article, nav
- C) header, section, div

▶ 解答と解説

問題1-2: 進捗バー構造穴埋め (10分)

```
<!-- 進捗表示 -->
<_____ class="progress-section">
  <h2><img alt="checkmark icon" data-bbox="183 601 203 616"/> 学習進捗</h2>
  <div class="progress-container">
    <div class="_____-bar">
      <div class="_____-fill" id="_____Fill"></div>
    </div>
    <p class="_____-text" id="_____Text">0 / 6 完了 (0%)</p>
  </div>
</_____>
```

考える問題:

1. 進捗表示全体を囲むタグは? (セマンティックHTMLを意識)
2. 進捗バーの外側divのclass名は?
3. 進捗バーの内側divのclass名は?
4. JavaScriptで操作するためのid名は?

ヒント:

- 外側は「枠」、内側は「実際の進捗」
- id名は JavaScript で使いやすい名前

▶ 💡 解答と解説

🧩 問題1-3: タスクリスト穴埋め (10分)

```
<!-- タスクリスト -->
<section class="tasks-section">
  <h2>✅ タスクリスト</h2>

  <div class="_____-item">
    <input type="_____" id="task1" class="_____-checkbox">
    <_____ for="_____">HTML基本構造を作成</label>
  </div>

  <!-- 🎯 チャレンジ: 残り5つのタスクを自分で書いてみよう! -->

</section>

<!-- 完了メッセージ -->
<div class="completion-message" id="_____Message">
  <h2>🎉 おめでとうございます!</h2>
  <p>全てのタスクが完了しました!</p>
</div>
```

🧠 自分で考える問題:

1. タスク1つ分のclass名は?
2. チェックボックスのtype属性は?
3. チェックボックスとテキストを関連付けるタグは?
4. 完了メッセージのid名は?

🎯 **チャレンジ:** 残り5つのタスクアイテムを自分で作ってください! (内容: "CSSで基本レイアウト", "グラデーション背景を追加", "ホバーエフェクト実装", "JavaScriptで進捗バー作成", "完了エフェクト追加")

▶ 💡 解答例

Step 2: CSS穴埋めチャレンジ (35分)

🧩 問題2-1: 基本設定穴埋め (10分)

```
/* 基本設定 */
_____ {
  margin: _____;
  padding: _____;
  box-sizing: _____-box;
}

body {
  font-family: sans-serif;
```

```
background: _____; /* グレー系の色 */
padding: _____px; /* 全体の余白 */
}
```

👉 ペアで話し合い:

1. 全要素を選択するセレクトは？
2. 要素の外側余白をリセットする値は？
3. `box-sizing` の値は？
4. 背景色は何色が良い？（色コードで）
5. 全体の余白は何px？

ヒント:

- リセットCSS の基本
- グレー系: `#f3f4f6`, `#e5e7eb`, `#gray` など

▶ 💡 解答と解説

🧩 問題2-2: カードレイアウト穴埋め (15分)

```
/* カード型レイアウト */
.main-header, _____ {
  background: _____;
  padding: _____rem;
  border-radius: _____px;
  margin-bottom: _____rem;
  box-shadow: _____ px rgba(0, 0, 0, _____);
}

/* 進捗バー */
.progress-bar {
  width: _____;
  height: _____px;
  background: #e5e7eb;
  border-radius: _____px;
  overflow: _____;
}

.progress-fill {
  height: _____;
  background: _____; /* 緑系の色 */
  width: _____%; /* 初期値 */
  transition: width _____s ease;
}
```

🧠 考える問題:

1. headerと同じスタイルを適用するセレクトは？
2. カードの背景色は？
3. `box-shadow` の構文は 水平 垂直 ぼかし 色 ですが、適切な値は？
4. 進捗バーの初期幅は何%？

5. アニメーション時間は何秒が自然？

💡 **実験してみよう：** 値を変更して効果を確認！

▶ 💡 解答例

✂️ **問題2-3: ホバーエフェクト穴埋め (10分)**

```
/* タスクアイテム */
.task-item {
  display: _____;
  align-items: _____;
  padding: 1rem;
  background: #f8fafc;
  border-radius: 8px;
  border: 2px solid #e5e7eb;
  cursor: _____;
  transition: _____ s ease;
}

/* 🎯 チャレンジ: ホバーエフェクトを自分で作ろう！ */
.task-item:hover {
  background: _____; /* 少し違う色に */
  border-color: _____; /* アクセントカラーに */
  transform: translateY(_____px); /* 上に移動 */
  box-shadow: _____px rgba(79, 70, 229, 0.2);
}
```

🎯 **ホバーエフェクト完全自作チャレンジ:**

1. `display` と `align-items` でどんなレイアウト？
2. マウスカースルの形は？
3. ホバー時の背景色は？（少し変化させる）
4. 何px上に移動させる？
5. 影の効果は？

👉 **ペアで実験：** 値を変更して一番良い効果を見つけよう！

▶ 💡 解答例

Step 3: JavaScript穴埋めチャレンジ (25分)

✂️ **問題3-1: 要素取得穴埋め (5分)**

```
document.addEventListener("_____", function () {
  // 要素を取得
  const taskCheckboxes = document._____("_____-checkbox");
  const progressFill = document._____("_____-Fill");
  const progressText = document._____("_____-Text");
});
```

```
const completionMessage = document._____("_____"Message");
});
```

 考える問題:

1. DOMが読み込まれた時のイベント名は？
2. 複数要素を取得するメソッドは？
3. ID で1つの要素を取得するメソッドは？
4. class名とid名の正確な名前は？

▶ 解答

 問題3-2: イベント処理ロジック穴埋め (10分)

```
// 各チェックボックスにイベントを追加
taskCheckboxes._____(function(checkbox) {
  checkbox.addEventListener("_____", function () {
    const taskItem = checkbox._____("task-item");

    if (checkbox._____) {
      taskItem.classList._____("completed");
    } else {
      taskItem.classList._____("completed");
    }

    _____(); // 進捗更新関数を呼び出し
  });
});
```

 ロジック理解チャレンジ:

1. 配列の各要素に処理を実行するメソッドは？
2. チェック状態変化を検知するイベントは？
3. 親要素を取得するメソッドは？
4. チェック状態を確認するプロパティは？
5. クラスを追加/削除するメソッドは？

▶ 解答

 問題3-3: 進捗計算完全自作チャレンジ (10分)

```
function updateProgress() {
  //  以下を自分で実装してみよう！

  // 1. 全タスク数を取得 (ヒント: taskCheckboxes.length)
  const totalTasks = _____;

  // 2. 完了したタスク数を取得 (ヒント: :checked セレクタ)
  const completedTasks = document.querySelectorAll("_____").length;

  // 3. パーセンテージを計算 (四捨五入)
```

```
const percentage = Math.round((_____ / _____) * 100);

// 4. 進捗バーの幅を更新
progressFill.style._____ = percentage + "%";

// 5. 進捗テキストを更新
progressText._____ = `${_____} / ${_____} 完了 (${_____}%)`;

// 6. 全完了時の判定
if (_____ === _____) {
  completionMessage.style.display = "_____";
} else {
  completionMessage.style.display = "_____";
}
}
```

 **完全自作問題:** この関数を自分で完成させてください！

考えるポイント:

1. 全タスク数の取得方法
2. 完了タスクの条件 (:checked)
3. パーセンテージ計算式
4. DOM要素のプロパティ更新
5. 条件分岐の判定

 **ペア作業:** 一緒に考えて実装しよう！

▶  解答例

Step 4: デバッグチャレンジ (20分)

問題4-1: バグ発見・修正チャレンジ (10分)

以下のコードにはバグがあります。見つけて修正してください！

```
<!-- HTML -->
<div class="task-item">
  <input type="checkbox" id="task1" class="task-check">
  <label for="task-1">HTML基本構造を作成</label>
</div>
```

```
/* CSS */
.task-item hover {
  background: #f1f5f9;
  transform: translateY(-2px);
}
```

```
// JavaScript
const checkboxes = document.querySelectorAll(".task-checkbox");
const progressText = document.getElementById("progress-text");

checkboxes.forEach(checkbox => {
  checkbox.addEventListener("click", function() {
    updateProgress();
  });
});
```

バグ発見ミッション:

1. HTMLの問題は？
2. CSSの問題は？
3. JavaScriptの問題は？

 ペアで話し合い：何が間違っているか見つけよう！

▶  バグと修正方法

問題4-2: 機能拡張チャレンジ (10分)

基本機能が完成したら、以下の機能を追加してみよう！

チャレンジミッション (難易度別) :

初級: 完了メッセージをクリックで閉じる機能

```
completionMessage.addEventListener("_____", function() {
  // ここに処理を書こう
});
```

中級: タスク完了時に背景色を変える機能

```
.task-item.completed {
  background: _____; /* 薄い緑 */
  border-color: _____; /* 緑 */
}

.task-item.completed label {
  text-decoration: _____; /* 取り消し線 */
  color: _____; /* グレー */
}
```

上級: 進捗バーの色をパーセンテージで変える機能

```
// ヒント: パーセンテージに応じて色を変更
if (percentage < 50) {
  progressFill.style.background = "_____"; // 赤系
} else if (percentage < 100) {
  progressFill.style.background = "_____"; // 黄系
}
```



```
} else {  
  progressFill.style.background = "_____"; // 緑系  
}
```

👉 ペアで挑戦：どのレベルまでできるか試してみよう！



穴埋め完了！おめでとうございます！



学習効果チェック

👉 ペアで話し合い：

1. 一番難しかった穴埋めは？その理由は？
2. 一番「なるほど！」と思った部分は？
3. 写経と穴埋め、どちらが理解しやすかった？



できるようになったこと

- ☒ HTMLの構造を理由と共に設計
- ☒ CSSプロパティの意味を理解して記述
- ☒ JavaScriptロジックを自分で構築
- ☒ バグを発見・修正する能力
- ☒ 機能を拡張する思考力



穴埋めサポートシステム

段階別ヒント提供

レベル1: 方向性ヒント

- 「〇〇について考えてみてください」
- 「完成形と比べて何が必要ですか？」

レベル2: 選択肢ヒント

- 「A, B, C のどれが適切でしょうか？」
- 「〇〇という方法はでしょうか？」

レベル3: 直接ヒント

- 「この部分は〇〇を使います」
- 「構文は『〇〇: △△;』です」

穴埋め特有のサポート

考える時間の確保

- 🕒 制限時間: 各問題に適切な時間設定
- 🗨️ ペア相談: わからない時は相談OK
- 💡 ヒント要求: 困ったら手を挙げる

答え合わせプロセス

1. 自分の答えを発表
2. 理由を説明
3. 正解と比較
4. なぜその答えなのか理解

よくある間違いパターン

- ✗ 「書けばいいと思った」 ✓ 「この理由でこの答えにした」
- ✗ 「なんとなく選んだ」 ✓ 「〇〇の効果を狙ってこれを選んだ」

穴埋め効果の検証

写経との違い:

- 📝 写経: コードを見て書く → 手の記憶
- 🧩 穴埋め: 理由を考えて書く → 頭の理解

能動性の向上:

- 🤔 「なぜこの値？」を常に考える
- 💡 「他の選択肢はダメ？」を検討する
- 🎯 「どんな効果を狙う？」を明確化

Think, Fill, Code! 🧩 穴埋めで理解を深めて、本当の実装力を身につけよう！